AVOID TRAPS

Document Trap

Your season-end basketball tournament starts next week, so you want to brush up on the rules. You upload your league's 200-page rulebook and ask: "How many fouls until I'm out of the game?"

ChatGPT answers: "Five fouls and you foul out."

Wait a second. In last year's tournament, you remember a player picking up five fouls and staying in the game. So you dig through the rulebook yourself. Regular season: five fouls, exactly what ChatGPT said. But near the end there's a special section for tournaments, and in those games players get six.

The AI pulled the standard limit and missed the exception. The answer wasn't made up. It was incomplete. **Document Trap is thinking 'uploaded' means 'fully read.'**

IT'S ABOUT THE CONTEXT WINDOW

As you learned earlier in the course, the model reads its full context window every time you send a message. Your 200-page rulebook equals around 100,000 tokens. That's more than some models will load at once, and even when it fits, the system still needs room for your conversation and its answer.

So when a long document doesn't fit, and often even when it does, the system runs a process on it instead:

1. **The document gets split into chunks, each a paragraph or two long.**
2. **Each chunk becomes tokens, then one meaning vector.**
   The chunk's text splits into tokens, just like every message you send. The model reads all of those tokens together and boils what the chunk is about down to one list of numbers: a single meaning vector for the whole chunk. It's the Embeddings idea, one size up: a point in meaning-space for a whole passage instead of a single token.
3. **Your question becomes a vector too. The closest chunks get loaded.**

Now you know what happened with the rulebook. It became a few hundred chunks. Your question pulled the chunks closest to it in meaning, and "how many fouls until I'm out of the game?" sits right next to the regular-season foul rules. The tournament section near the back is about tournaments first and fouls second, so it didn't make the cut.

RETRIEVAL

There's a name for what just happened: **retrieval**. When the system can't load everything, it retrieves the pieces that match your question. Done well, this finds you a specific answer in a 200-page rulebook in seconds. Done poorly, the wrong pieces get pulled, and the model answers from incomplete evidence.

And you've met it before: this is the R in RAG from the Hallucination lesson, searching your file instead of the web.

WHAT YOU CAN DO

You can steer what the model retrieves. Four moves help, and all four share one idea: make the right chunks easy to find. The more specific your question, the more reliable the answer.

1. **Point to the section by name**
   Include keywords from the document in your question. The system uses them to find matching chunks.
2. **Ask one question at a time**
   Multi-part questions force the system to retrieve different chunks for different parts. One focused question per turn gives retrieval the best shot.
3. **Share only what's relevant**
   Paste the paragraphs that matter into your message, or upload just the chapter instead of the whole book. Less to search means the right chunks are more likely to make the cut, and a pasted section skips the retrieval lottery entirely.
4. **Ask the AI to quote**
   Add 'quote the exact section you're basing this on' to your question. If the quote is missing or doesn't match the document, retrieval probably failed.

This trap doesn't stay in basketball. The documents that run your life only get longer from here: apartment leases, employment contracts, insurance policies, financial aid letters. Uploading one and asking AI what it says will be the natural move, and it's a good one. Just remember the rulebook: the clause that changes everything is usually an exception near the back, exactly the kind of chunk that doesn't make the cut.

It answers from what it retrieved.

Not from having read the whole thing.